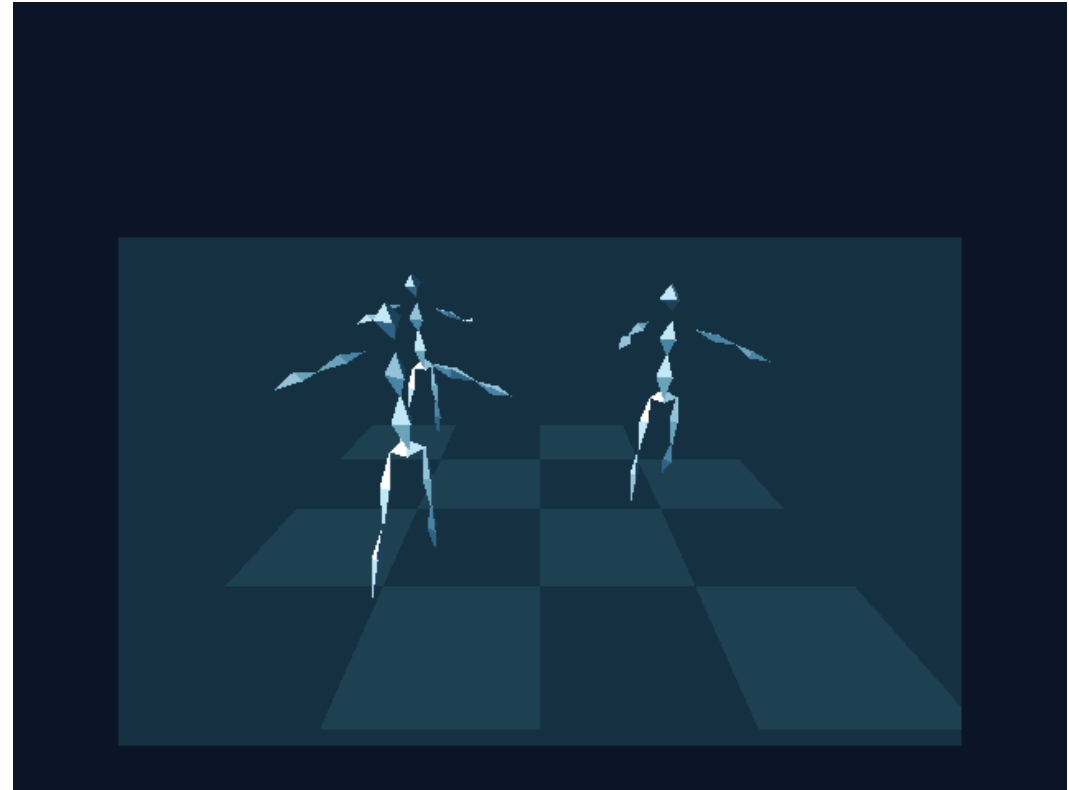


# BVH motion, baked for MSX

From joint hierarchy to low-poly dance on V9968 + geo3d

Learn three things:

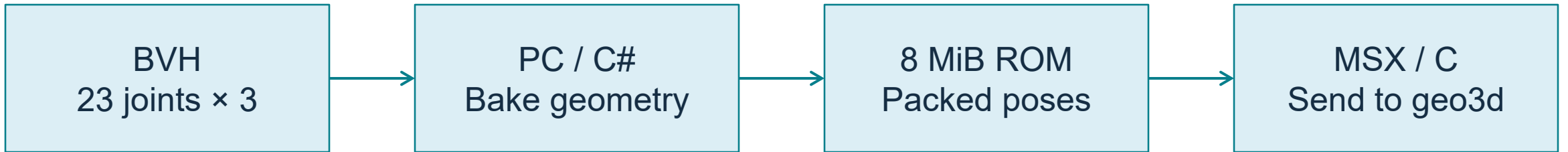
1. What BVH contains
2. What baking computes
3. What geo3d does at playback



**SCREEN 7 FIL · 512 × 424 · 16 colors**

# Baking moves repeated work to the PC

Precompute geometry once; choose and draw a pose during playback.



PC: parse text, compose joint transforms, build meshes, compute normals.

MSX: choose time, read a ROM bank, set the camera, upload geometry.

# BVH stores a skeleton and motion channels

It does not contain a textured character model.

## HIERARCHY

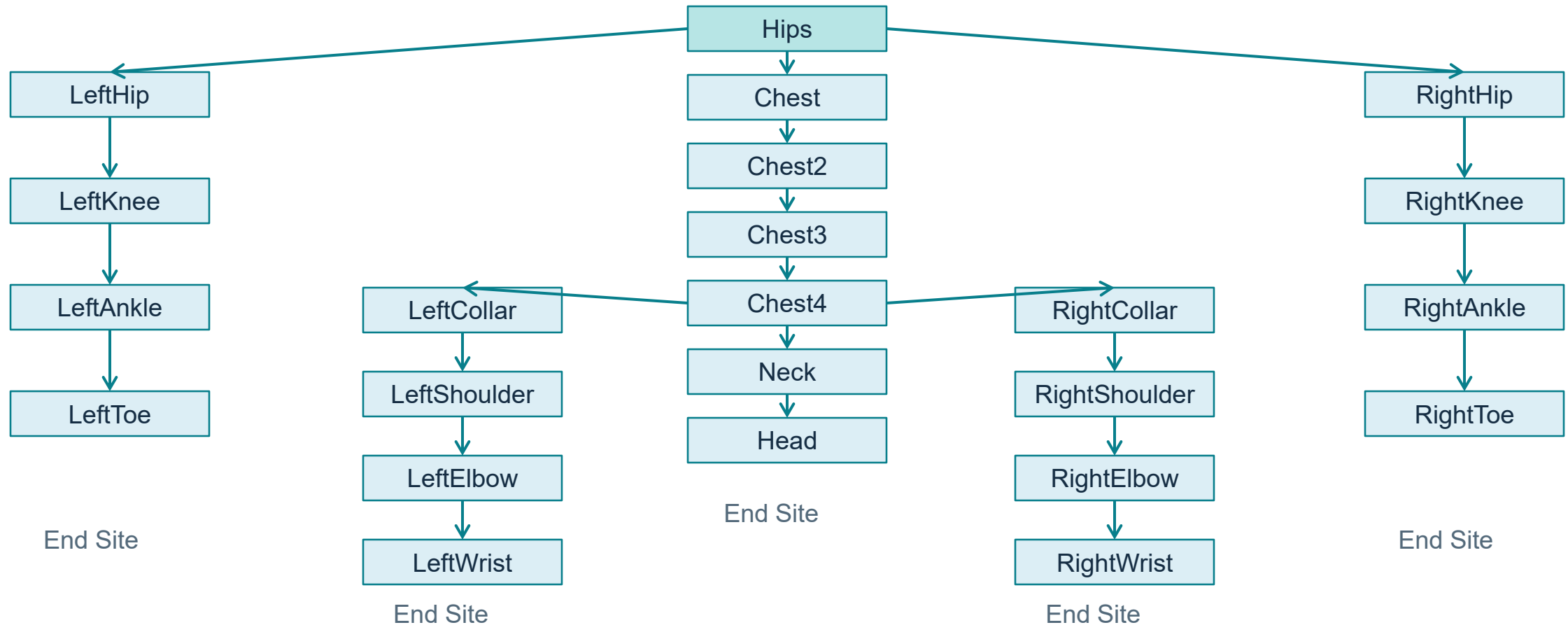
ROOT / JOINT: parent and child nodes  
OFFSET: displacement from the parent  
CHANNELS: the order of values in each frame  
End Site: an endpoint, with no motion channels

## MOTION

$2820 \text{ frames} \times 0.025 \text{ s} = 70.5 \text{ s}$   
Each row lists position / rotation channel values.  
All three clips have the same expected layout.

# The actual 23-joint hierarchy

Arrows mean “parent → child”; End Sites are endpoints, not joints.



# Read rotations in the declared order

This BVH declares Yrotation → Xrotation → Zrotation.

```
for (int k = 0; k < j.channels.Count; k++)  
    if (j.types[k].EndsWith("rotation"))  
        local = Mul(local, Rotate(j.types[k],  
            clip.motion[frame][j.channels[k]]));
```

OFFSET stays fixed.  
Channel values change  
each frame.

A rotation around Y  
followed by X generally  
differs from X followed by Y.

**L = Translation × Ry × Rx × Rz**

# A child inherits the parent's transform

Forward kinematics: walk from the root toward the endpoints.



```
world[i] = j.parent < 0 ? local  
          : Mul(world[j.parent], local);
```

$$W_{child} = W_{parent} \times L_{child}$$

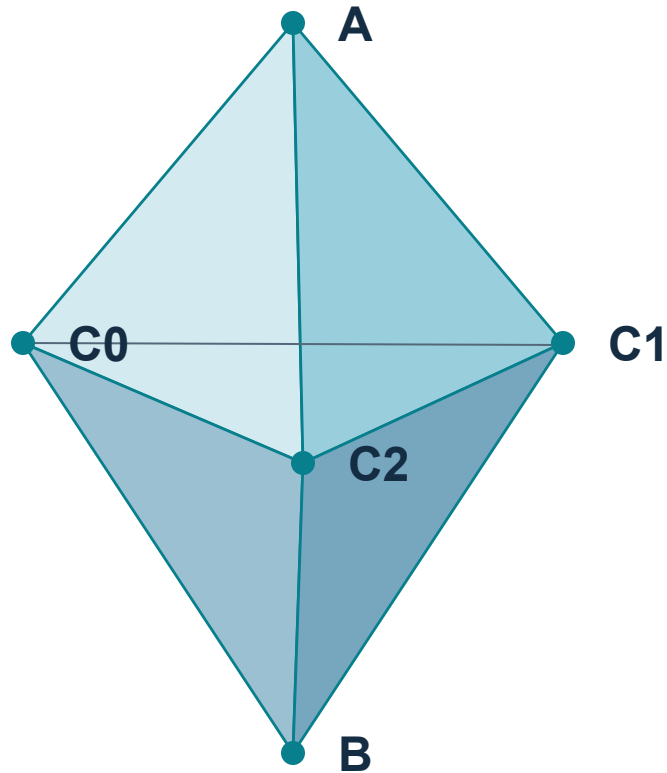
Turn the shoulder: elbow and wrist move together.

Turn the elbow: only its descendants follow.

**Point(world[j], position)** gives a position in the common world space.

# Turn each selected bone into a small solid

A triangular bipyramid: 5 vertices, 6 triangular faces.



12 parts per dancer  
= 60 vertices / 72 faces

3 dancers  
= 180 vertices / 216 faces

The surface is rigid per part.  
No smooth skinning is performed.

# Compute outward normals before playback

Normals tell geo3d which way a face points for lighting.

```
V n = (p.vertices[ib] - p.vertices[ia])  
      .Cross(p.vertices[ic] - p.vertices[ia]);  
p.faces.Add(new Face {  
    a = ia, b = ib, c = ic, n = n.Unit()  
});
```

Cross product →  
perpendicular direction  
Normalize → length 1

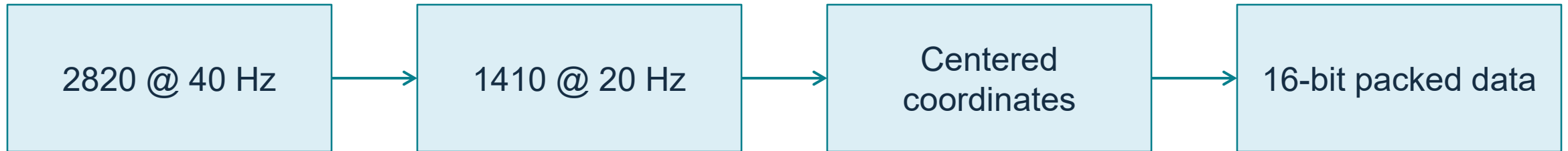
Stored normal: signed Q2.14  
1.0 becomes 16384.

Coordinates: signed 16-bit  
integers

**The omitted winding check flips inward-facing normals.**

# Bake 40 Hz input into 20 Hz poses

Sampling, centering and quantization happen on the PC.



```
Body(p, clips[actor], f * 2);  
V center = (lo + hi) * .5;  
V r = q - center;  
w.Write(Quant(r.x));  
w.Write(Quant(r.y));  
w.Write(Quant(r.z));
```

Read every second source frame.  
Duration stays 70.5 seconds.

Center uses the whole sequence's  
bounds, so the stage does not recenter  
every frame.  
No pose interpolation is implemented.

# One pose occupies 3456 bytes

The file is already arranged for geo3d's vertex and face streams.

## VERTICES

$$180 \times 6 = 1080 \text{ bytes}$$

## FACES

$$216 \times 11 = 2376 \text{ bytes}$$

Vertex / 6 bytes

X, Y, Z: signed int16

Little-endian word order

Face / 11 bytes

I0, I1, I2, I3: four uint8 indices

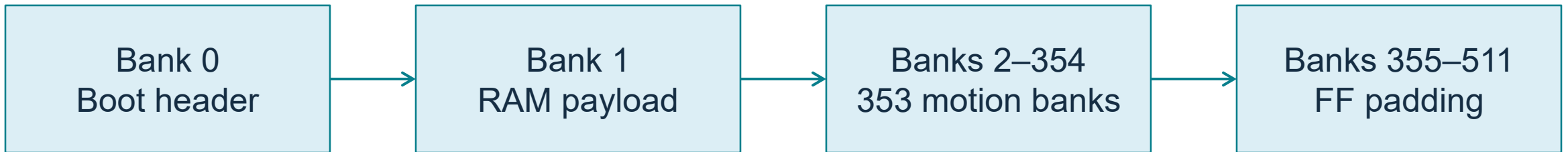
NX, NY, NZ: three Q2.14 int16 normals

BASE: one palette index

**Triangle (a, b, c) is sent as the quad record (a, b, c, c).**

# Four poses fit into each 16 KiB ROM bank

ASCII16-X extends bank selection beyond 256 banks.



A motion bank  
 $4 \times 3456 = 13824$  bytes  
+ 2560 bytes of padding

Last motion bank: only 2 poses

```
unsigned int bank =  
    MOTION_BANK + frame / 4;  
*(volatile unsigned char *)  
    (0x6000 | (bank & 0x0f00)) =  
    (unsigned char)bank;  
pose = (const unsigned char *)  
    (0x4000 + (frame & 3) * FRAME_BYTES);
```

# Execute in RAM; read motion from ROM

Changing a data bank must not replace the running code.

|           |                                       |                                                                   |
|-----------|---------------------------------------|-------------------------------------------------------------------|
| 0000–3FFF | BIOS                                  | At least 64 KiB main RAM<br>256 KiB VRAM                          |
| 4000–7FFF | ROM motion window                     |                                                                   |
| 8400–A06B | C code + variables / work area        | Interrupt handling stays in RAM and does not switch motion banks. |
| E000–E2xx | IM2 table / interrupt / loader helper |                                                                   |
| F300 ↓    | Stack (grows downward)                | This viewer reads one selected pose at a time.                    |

# CPU, geo3d and V9968 have different jobs

geo3d generates drawing commands; V9968 writes the pixels.



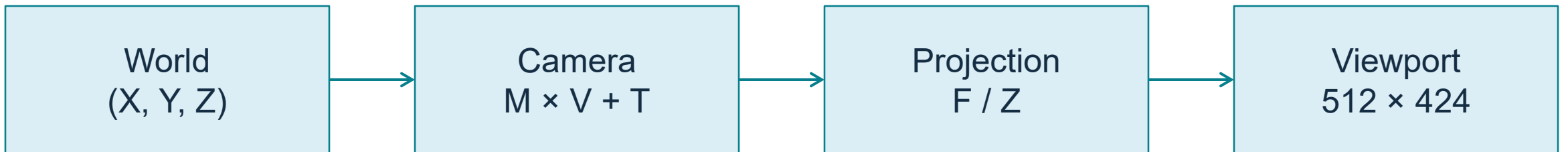
Time / input / PSG  
ROM bank selection  
Camera and uploads  
Clear / page display

Transform / perspective  
Face culling / shading  
Depth ordering  
LINE command generation

Command engine  
VRAM pixel writes  
Palette / scanout  
SCREEN 7 FIL display

# Projection turns 3D positions into pixels

The camera matrix and translation are sent separately from the mesh.



```
geo_index = 0x18;  
word(SCREEN_FOCAL);           // 320  
word(SCREEN_WIDTH / 2);       // 256  
word(SCREEN_HEIGHT / 2);      // 212  
word(48);                     // near plane
```

$$SX = CX + X \times F / Z$$
$$SY = CY - Y \times F / Z$$

World Y points upward.  
Pixel Y points downward.  
Farther objects look smaller.

# Upload streams, then issue RUN

This application uses geo3d index 9Dh and data 9Fh.

```
reg(0x40, 0);  
geo_index = 0x50;  
stream(v, (unsigned int)nv * 6);  
reg(0x58, 0);  
geo_index = 0x52;  
stream(f, (unsigned int)nf * 11);
```

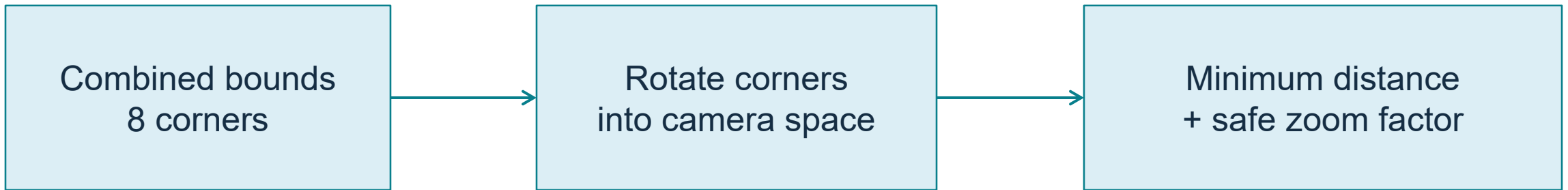
```
reg(0x42, nv); reg(0x59, nf);  
reg(0x48, 3);  
wait_geo();
```

40h / 58h: write pointers  
50h: vertex stream  
52h: face stream  
42h / 59h: counts  
48h = 3: RUN + filled faces

Do not change configuration or VDP drawing registers while RUN is busy.

# Keep the whole visible pose inside the frame

Fit the bounds of dancers, trails and reflected geometry together.



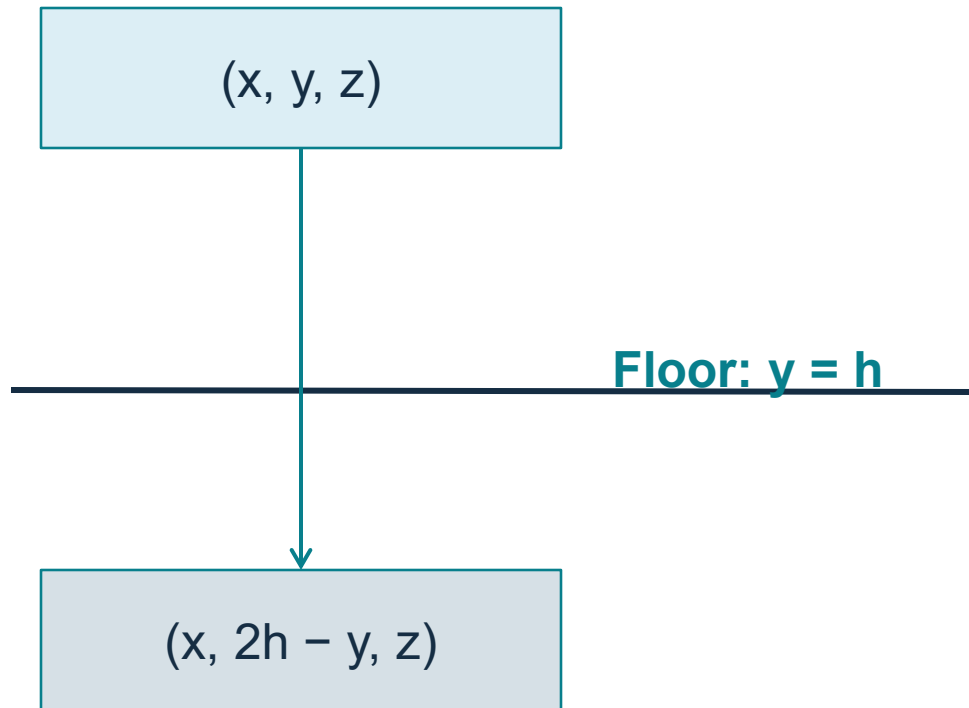
Test projected X / Y extents and near-plane clearance.

The closest allowed view is 100%; zooming to 200% moves the camera farther away.

Camera yaw is limited to  $\pm 45^\circ$ .

# A reflection is mirrored geometry

Mirror positions, vertex winding and normals as a set.



$$h = \text{FLOOR\_Y} = -174$$

Normal:  $(nx, ny, nz) \rightarrow (nx, -ny, nz)$

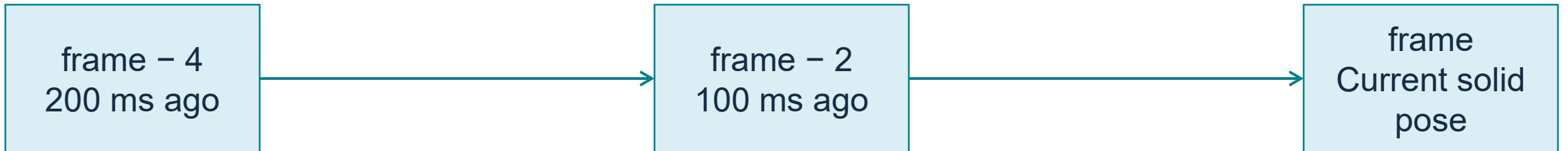
Face order:  $(a,b,c,c) \rightarrow (a,c,b,b)$

Use a darker palette range.

This is a second geometry pass, not physical reflection or alpha blending.

# Motion trails reuse older poses cheaply

Draw bone endpoints as lines instead of copying filled meshes.



Each bone contributes its first two stored vertices.

12 lines per actor; 36 lines for all three.

Two discrete colors give a fading impression.

This is a motion-trail effect, not continuous motion blur.

# Playback time is independent of render speed

A 60 Hz VBlank clock selects a 20 Hz motion pose.

```
now = clock_ticks();  
elapsed = now - previous;  
previous = now;  
playback_ticks += elapsed;  
motion_frame = playback_ticks / 3;
```

1 pose = 3 VBlank ticks  
1410 poses = 4230 ticks = 70.5 s

Slow rendering skips poses.  
It does not intentionally stretch the dance.

Last 0.5 s: palette + music fade.

**Excerpt: pause, end-of-clip and clamping checks are omitted.**

# SCREEN 7 FIL has two register backends

Choose the ROM for the V9968 implementation, not just the CPU.

MOTION.rom

Legacy openMSX FIL mapping  
R20 = 11h / R21 = 7Ah  
FIL: R21 bit 6  
Copy Y512 → Y0 for presentation

Emulator playback verified  
on R800 / Z80.

MOTION-native.rom

Current FPGA FIL mapping  
R20 = 31h / R21 = 3Ah  
FIL: R20 bit 5  
Display pages Y0 / Y512

Build verified;  
hardware operation untested.

# Where to read and what to change next

Start with one responsibility at a time.

`tools/BvhBake.cs`

BVH parser → transforms → mesh → ROM

`src/scene.c`

Bank reading, camera, geo3d, visual effects

`src/main.c`

Input, motion clock, pause and restart

`src/video.c`

SCREEN 7 FIL, palette, pages, white fade

`src/music.c`

Original looping PSG soundtrack

# Sources, credits and limits

Independent, unofficial motion viewer. Source review: 2026-10-03.

## BVH: Perfume global site project #001

<https://perfume-global.com/web/2012/03/perfume-global-site-project-001/>

## geo3d: Alex Moncks / V9968\_Cartridge

[https://github.com/alexmoncks/V9968\\_Cartridge/tree/main/geo3d](https://github.com/alexmoncks/V9968_Cartridge/tree/main/geo3d)

Original meshes and PSG music; the official MP3 is not used.

No license is declared for original code. GPL-2.0 applies only to the two machine XML files.

Performance and hardware compatibility need separate measurements.